Accord — Architecture Document

Status

Draft v0.1

Purpose

This document maps the Accord thesis, PRD, and protocol spec into a concrete implementation architecture.

It answers:

- what components exist in v1,
- how they interact,
- what data flows through the system,
- how local-first operation should work,
- where the trust boundaries are,
- and how the repository should be structured for implementation.

This is a **v1 architecture**, intentionally optimized for clarity, inspectability, and disciplined scope.

---

1. Architecture Goals

Accord's first architecture should satisfy six goals:

1.1 Local-first operation

A developer should be able to run the full system locally without cloud infrastructure.

1.2 Protocol-driven implementation

All major runtime behavior should be built around the protocol objects, not hidden ad hoc state.

1.3 Fail-closed execution

If policy or approval requirements are not satisfied, the system must block execution.

1.4 Clear trust boundaries

The boundary between proposing, approving, and executing actions must be explicit.

1.5 SDK-first adoption

The primary entry point for external developers should be a clean TypeScript SDK.

1.6 Small but expandable core

The architecture should be minimal for v1 while leaving clean extension points for adapters and future storage or policy evolution.

---

2. High-Level Component Model

Accord v1 should be built from six primary components:

1. **Protocol Layer**
2. **SDK Layer**
3. **Accord Server**
4. **Policy Engine**
5. **Approval Console**
6. **Execution Adapters / Sample Apps**

These components form the local runtime system.

---

3. Component Responsibilities

3.1 Protocol Layer

This is the structural foundation of the system.

It defines: - protocol object types, - schemas and validation rules, - state transitions, - and object relationships.

The protocol layer should be implementation-neutral as much as possible.

Responsibilities

- object schemas
- type definitions
- validation contracts
- status enums
- shared constants
- serialization format

Non-responsibilities

- persistence
- UI
- transport
- execution

3.2 SDK Layer

The SDK is the main developer-facing interface.

It should make the common Accord flows ergonomic without hiding the protocol model.

Responsibilities

- create scope requests
- create context manifests
- create action proposals
- fetch decision / approval state
- verify approval receipts before execution
- provide typed client utilities

Non-responsibilities

- storing raw user memory itself
- executing external side effects directly by default
- replacing agent frameworks

Example SDK Surface

The SDK should eventually expose primitives like: - `requestScope()` - `getGrantedScope()` - `publishContextManifest()` - `proposeAction()` - `awaitDecision()` - `verifyReceipt()` - `revokeScope()`

These names are placeholders, but the overall feel should be simple and unsurprising.

3.3 Accord Server

The Accord Server is the local control-plane runtime.

It should be a small local service responsible for receiving protocol objects, validating them, evaluating policy, persisting state, and serving the console.

Responsibilities

- accept protocol requests from SDK clients
- validate object shapes
- persist runtime objects
- evaluate policies
- create approval tasks
- issue receipts
- record audit events
- expose local APIs for console and SDK

Non-responsibilities

- acting as a general orchestration engine
- becoming a hosted SaaS in v1
- replacing external data stores or memory engines

The server is the authoritative state manager for the local Accord runtime.

3.4 Policy Engine

The policy engine determines the enforcement mode for scope and action flows.

Responsibilities

- evaluate scope requests
- evaluate action proposals
- classify allow / require_approval / deny
- enforce expiry and revocation checks
- fail closed when required objects are invalid or stale

Non-responsibilities

- machine learning risk scoring in v1
- adaptive behavioral prediction
- complex enterprise policy hierarchies

The policy engine should remain boring and predictable in v1.

3.5 Approval Console

The console is a minimal local UI for human interaction.

It is not the product itself. It is the reference inspection and approval surface.

Responsibilities

- show pending action proposals
- show context manifests attached to those proposals
- allow approve / deny actions
- show active scopes
- show scope expiry and revocation state
- expose audit history

Non-responsibilities

- being a polished consumer product
- replacing host application UIs
- providing broad analytics dashboards in v1

The console's job is legibility, not visual ambition.

3.6 Execution Adapters / Sample Apps

Execution adapters connect Accord's control-plane decisions to actual side effects.

Responsibilities

- call the SDK from a real app flow
- block execution until Accord requirements are satisfied
- verify receipts before side effects
- demonstrate how Accord fits into real systems

Initial Targets

- local coding agent sample
- local research-to-publish sample
- optional lightweight MCP-facing adapter

Non-responsibilities

- abstracting every agent framework in v1
- supporting every external platform early

---

4. High-Level Runtime Flow

The core runtime path should look like this:

1. an application or agent requests scope,
2. the server validates the request,

3. the policy engine returns allow / require_approval / deny,
4. a granted scope is issued if appropriate,
5. the application uses permitted context,
6. the application publishes a context manifest,
7. the application proposes an action,
8. the policy engine evaluates the proposal,
9. the console presents the proposal if approval is required,
10. an approval receipt is issued or denied,
11. the executor verifies the receipt,
12. the action runs only if conditions are satisfied,
13. all state transitions emit audit records.

This flow is the architecture's backbone.

---

5. Trust Boundaries

Accord is fundamentally about boundaries, so the architecture should name them clearly.

Boundary A — Context Access Boundary

This is where requested access to user context is evaluated.

Questions answered here: - who is asking? - what categories are requested? - for what purpose? - how long should access last?

Relevant objects: - `ScopeRequest` - `GrantedScope`

Boundary B — Context Disclosure Boundary

This is where the application declares what context it actually used.

Questions answered here: - which entries influenced the action? - how are those entries summarized? - what provenance or source do they have?

Relevant object: - `ContextManifest`

Boundary C — Action Approval Boundary

This is where a side effect is paused until policy and approval conditions are satisfied.

Questions answered here: - what is the action? - what target will it affect? - what risk level applies? - is human approval required?

Relevant objects: - `ActionProposal` - `ApprovalReceipt`

Boundary D — Execution Boundary

This is where the actual side effect happens.

Accord itself should not silently cross this boundary.

It should only be crossed when: - scope is valid, - manifest is present, - proposal is valid, - policy permits, - and approval is satisfied if required.

Boundary E — Audit Boundary

This is where every state transition becomes durable and inspectable.

Relevant object: - `AuditRecord`

These boundaries are more important than the UI.

---

6. Proposed Monorepo Structure

A TypeScript monorepo is the strongest fit for v1.

Recommended structure:

```
accord/
  README.md
  package.json
  pnpm-workspace.yaml
  docs/
    00-project-thesis.md
    01-research-brief.md
    02-prd.md
    03-protocol-spec.md
    04-architecture.md
    05-threat-model.md
    06-roadmap.md
    adr/
  packages/
    protocol/
    sdk/
    policy-engine/
    server/
    console/
    adapter-mcp/
  examples/
```

```
    coding-agent/
    research-publisher/
  tooling/
    scripts/
    config/
```

Package Intent

- `protocol/` — shared types, schemas, validators
- `sdk/` — developer client library
- `policy-engine/` — rules and decision logic
- `server/` — local service and APIs
- `console/` — local React UI
- `adapter-mcp/` — optional adapter layer demonstrating ecosystem fit
- `examples/` — real usage flows

This structure keeps the repo legible for contributors.

---

7. Storage Architecture

For v1, storage should be simple, local, and inspectable.

Recommended v1 Storage Model

Primary choice

SQLite for local structured persistence

Why: - easy local setup - strong enough relational model for linked protocol objects - inspectable with standard tools - low operational complexity

What should be stored

At minimum, persistent tables or equivalent collections for: - scope requests - granted scopes - context manifests - action proposals - approval receipts - revocation events - audit records - policy configs

Why not a vector database?

Because Accord is not trying to be a memory retrieval engine in v1.

Why not Postgres first?

Postgres is fine later, but SQLite keeps the local-first story simpler.

---

8. API Architecture

The server should expose a minimal local API surface.

A simple HTTP JSON API is enough for v1.

Candidate Endpoint Groups

Scope Endpoints

- create scope request
- list active scopes
- revoke scope

Manifest Endpoints

- publish context manifest
- fetch manifest details

Proposal Endpoints

- create action proposal
- fetch proposal
- list pending proposals

Approval Endpoints

- approve proposal
- deny proposal
- fetch receipt

Audit Endpoints

- list audit records
- fetch records by object

Policy Endpoints

- get policy config
- update local policy rules

The exact routes can stay small and implementation-driven. The main goal is preserving the protocol model cleanly.

---

9. Internal Server Modules

Inside the server package, I would keep the architecture modular but not over-engineered.

Suggested internal modules:

- **validation module**
- schema checks
- required field enforcement
- **scope service**
- create and resolve scope requests
- issue granted scopes
- expiry checks
- **manifest service**
- validate and store manifests
- **proposal service**
- create and manage action proposals
- status transitions
- **approval service**
- create decisions
- issue receipts
- enforce single-use or expiry conditions
- **policy service**
- evaluate requests and proposals
- **audit service**
- append audit records
- query history
- **storage layer**
- database access
- migrations

This is enough structure without turning the project into ceremony.

---

10. Policy Engine Architecture

The v1 policy engine should be rules-based.

Recommended Approach

A lightweight local rules system, likely JSON- or file-configured first.

For example, rules can inspect: - requester ID - context category - action type - target type - risk level - scope status - expiry state

And return: - `allow` - `require_approval` - `deny`

Why not build a DSL first?

Because that is exciting but premature.

Why not hardcode everything?

Because developers should be able to inspect and tweak behavior locally.

So the sweet spot is: - structured config, - predictable matching, - simple precedence.

---

11. Console Architecture

The console should be intentionally small and task-focused.

Core Views

View 1 — Pending Approvals

Shows action proposals waiting on review.

For each proposal, show: - summary - action type - target - risk level - requester - attached context manifest summary - approve / deny controls

View 2 — Active Scopes

Shows currently valid granted scopes.

For each scope, show: - requester - categories - purpose - issued time - expiry time - revoke action

View 3 — Audit History

Shows append-only history across: - requests - grants - proposals - approvals - revocations

View 4 — Proposal Detail

Shows the proposal and related context manifest clearly enough for a human decision.

This UI is about operational clarity, not polish.

---

12. SDK Integration Flow

The SDK should support a flow like this:

```
const scope = await accord.requestScope({...})
const manifest = await accord.publishContextManifest({...})
const proposal = await accord.proposeAction({...})
const decision = await accord.awaitDecision(proposal.id)
const verified = await accord.verifyReceipt(decision.receiptId)
if (verified.ok) {
  await executeSideEffect()
}
```

The real API may differ, but the shape should feel this simple.

SDK Design Principles

- typed responses
- minimal required setup
- clear error states
- no hidden execution
- easy local configuration

---

13. Execution Model

Accord should never blur into silent executor logic.

The execution model should be explicit:

1. host application prepares action,
2. host application submits proposal,
3. Accord evaluates and possibly pauses,
4. host application receives approval result,

5. host application calls its own executor only after verification.

This preserves a clear boundary between **control plane** and **side effect plane**.

That separation is important both architecturally and philosophically.

---

14. Example Sequence — Coding Agent

Flow

1. coding agent requests access to `project_preferences` and `git_rules`
2. server validates and grants scope
3. agent reads allowed preferences from local context store
4. agent drafts a file edit and commit plan
5. agent publishes a context manifest describing what rules it used
6. agent proposes `git.commit`
7. policy marks `git.commit` as `require_approval`
8. console shows proposal and manifest
9. user approves
10. receipt is issued
11. executor verifies receipt
12. commit runs
13. audit events are recorded throughout

This is the canonical v1 story.

---

15. Observability in v1

We should keep observability modest but real.

Minimum Signals

- request counts by object type
- policy decision counts
- pending approval counts
- approval latency
- denied / expired / revoked counts
- execution verification failures

These can begin as local structured logs plus a simple diagnostics view if needed.

We do not need full production telemetry architecture in v1.

---

16. Security and Safety Posture in v1

The system should make a few strong choices early:

- local-only by default
- no hidden auto-execution for approval-required actions
- append-only audit log behavior in the reference implementation
- explicit expiry and revocation checks
- executor must verify before side effects

This is not a full security platform, but the architecture should reinforce trustworthy behavior.

---

17. Deferred Architectural Concerns

The following are deliberately postponed:

- cloud multi-tenancy
- distributed approval workflows
- identity federation
- multi-device sync
- encrypted remote storage
- delegated approvals
- plugin marketplace architecture
- policy inheritance trees
- advanced analytics
- multi-language SDKs beyond the first core path

These will only be worth solving after the local control model feels solid.

---

18. Architecture Risks

Risk 1 — Overbuilding the server

If the server becomes an orchestration platform, the project will lose clarity.

Risk 2 — Underbuilding the protocol boundary

If proposal, receipt, and manifest flows are not explicit in code, the system will collapse into ad hoc app logic.

Risk 3 — Too much UI, too early

If the console becomes the focus, the SDK and spec will suffer.

Risk 4 — Too many adapters early

Trying to support every agent ecosystem too soon will slow core product maturity.

Risk 5 — Storage abstraction too early

Inventing pluggable storage layers before the data model is stable will add noise.

---

19. Recommended Implementation Order

The architecture suggests this build order:

1. protocol package
2. local validation and shared types
3. storage layer and migrations
4. server core modules
5. rules-based policy engine
6. SDK client
7. minimal console
8. first example integration
9. second example integration
10. optional MCP adapter

This order keeps the system grounded in the protocol rather than UI-first development.

---

20. Closing Statement

Accord’s architecture should feel like a clean local control plane for approval-aware personalization.

It should not feel like a giant framework, a half-built assistant, or a speculative platform.

If we implement the protocol cleanly, keep the server small, and make the SDK pleasant, the architecture will do exactly what it needs to do: make context use explicit, make action approval unavoidable where it matters, and leave behind an inspectable record.